

ספר הפרויקט — StorePilot

שער

שם הפרויקט: StorePilot — מערכת ניהול חנות ואפליקציית קניות

שם התלמיד: _____

ת.ז. התלמיד: _____

שם בית הספר: _____

שם המנחה: _____

שם החלופה: תכנון ותכנות מערכות — חלופה טלפונים חכמים (5 יח"ל)

תאריך ההגשה: _____

תוכן עניינים

- 1. [מבוא](#)
 - 2. [מבנה / ארכיטקטורה של הפרויקט](#)
 - 3. [מימוש הפרויקט](#)
 - 4. [מדריך למשתמש](#)
 - 5. [סיכום אישי / רפלקציה](#)
 - 6. [ביבליוגרפיה](#)
 - 7. [נספחים](#)
-

מבוא

הרקע לפרויקט

שם הפרויקט: StorePilot

תיאור קצר: StorePilot היא אפליקציית אנדרואיד דו-תפקידית המשלבת בתוכה שני עולמות: מערכת ניהול חנות מלאה עבור בעל החנות והעובדים, ואפליקציית קניות אונליין עבור הלקוחות. המערכת בנויה סביב ארכיטקטורת MVVM (Model-View-ViewModel), משתמשת בבסיס נתונים מקומי (Room (SQLite ובמקביל מסונכרנת ענן באמצעות Firebase Authentication ו-Firebase-1.

קהל היעד: - בעלי חנויות קטנות ובינוניות המעוניינים בניהול דיגיטלי של המלאי, המכירות, המשימות והעובדים. - מנהלי חנויות ועובדים הזקוקים לכלי עבודה יומיומי. - לקוחות פרטיים המעוניינים לקנות מהחנות באמצעות אפליקציה נוחה ומאובטחת.

הסיבות לבחירת הנושא: 1. שילוב של תחומים שונים — ניהול עסקי, אבטחה (הצפנת סיסמאות, אימות זהות בעל החנות באמצעות קוד סודי), חוויית משתמש מודרנית, וענן. 2. רלוונטיות חזקה — חנויות קטנות רבות עדיין מנהלות מלאי ומכירות ידנית. 3. הזדמנות לבטא מגוון רחב של נושאים מתקדמים: Firebase, AlarmManager, Notifications, Fragments, Threads, Room, MVVM ועוד.

תהליך המחקר

מחקר על תחום הידע: נערכה סקירה של מערכות ניהול חנות קיימות (POS systems) כמו Shopify, Square ו-Lightspeed. נמצא כי רוב המערכות הללו יקרות ומורכבות עבור חנויות קטנות.

בדיקת אפליקציות קיימות: נבדקו אפליקציות מסחר אלקטרוני (E-commerce) כגון Amazon, Shopify Customer App, Wix Owner — והוסקה המסקנה ששילוב של שתי הזוויות (ניהול + קנייה) באפליקציה אחת אינו נפוץ.

סקירת המצב הקיים בשוק: רוב המערכות לחנויות מציעות רק את צד הניהול או רק את צד הלקוח, מה שמחייב את בעל החנות לרכוש שני פתרונות נפרדים. StorePilot מציעה פתרון מאוחד.

אתגרים מרכזיים

1. **ניהול הרשאות מורכב** — ארבעה תפקידים (Owner, Store Manager, Shift Manager, Employee) בנוסף לתפקיד Customer, כאשר לכל אחד מהם הרשאות שונות.
2. **אימות זהות בעל החנות** — מנגנון קוד סודי (777) שמופיע כדיאלוג בעת בחירת תפקיד Owner.
3. **סנכרון נתונים בין Room מקומי ל-Firebase ענן** — שמירה כפולה בעת כל פעולת insert/update/delete.
4. **התראות בזמן אמת** — בעת ירידת מלאי מתחת לסף, על המערכת לשלוח notification למשתמש גם כשהאפליקציה ברקע.
5. **תצוגות LiveData מקושרות** — תצוגות שמשלבות מספר מקורות נתונים (מועדפים + מוצרים, עגלה + מוצרים) הצריכו ניהול מדויק של observers.

חידושים והתאמות

- **שמירה כפולה (Dual Write)** — כל פעולת DB נכתבת גם ל-Room וגם ל-Firebase, מה שמאפשר עבודה offline ובמקביל סנכרון לענן.
- **קוד אימות Owner** — שיטה חדשנית לאימות בעל החנות לפני הרשמה.
- **AlarmManager תקופתי** — בדיקת מלאי כל שעה ושליחת notification אוטומטי.

תיאור תחום הידע

- אובייקטים נחוצים:** - User — משתמש המערכת (Owner / Store)
- Product (Manager / Shift Manager / Employee / Customer)
- Sale — רישום מכירה - Purchase — רישום רכישה מספק
- Order — הזמנת לקוח - OrderItem — פריט בהזמנה - CartItem
- Favorite — מוצר במועדפים של לקוח - Task

— משימה לעובד - Season — עונה (קולקציה זמנית) - VideoMetric —
מדד שיווקי לסרטון - SupportMessage — הודעת צ'אט תמיכה

סוגי נתונים: - מחרוזות (String) — שמות, אימיילים, סיסמאות מוצפנות,
כתובות - מספרים שלמים (int) — מזהי שורה, כמויות - מספרים עשרוניים
(double) — מחירים, סכומים - חותמות זמן (long) — תאריכים בפורמט
UNIX timestamp - בוליאניים (boolean) — סטטוסים שונים

ייצוג מידע: - **ArrayList** — רשימות דינמיות של אובייקטים בתצוגות
(RecyclerView) - **LiveData** < — תצפית ריאקטיבית על נתוני Room -
טבלאות Room — אחסון מקומי בנוסחת SQLite עם relations וקלידים
זרים - **Firestore Collections** — אחסון ענן בנוסחת מסמכים מקוננים
(NoSQL)

פעולות על המידע: - הוספה (insert) — כל ViewModel קורא
ל-Repository שמבצע insert גם ל-Room וגם ל-Firebase - קריאה (read)
— LiveData מ-Room לתצוגה מיידית - עדכון (update) — דרך MVVM
זרימה: DAO → Repository → ViewModel → View +
FirestoreManager - מחיקה (delete) — באישור משתמש בלבד, נמחק
משתי המקורות

מבנה / ארכיטקטורה של הפרויקט

תיאור מסך הפתיחה — WelcomeActivity

מסך הפתיחה הוא נקודת כניסה למשתמשים חדשים ולכאלה שעדיין לא
נכנסו. הוא בודק תחילה אם המשתמש מחובר כבר
(SessionManager.isLoggedIn) — אם כן, מנתב אותו ישירות לפי תפקידו
(Customer → CustomerMainActivity; Owner/Manager/Employee)
→ MainActivity). אם לא, מציג שני כפתורים: **Sign In** ו-**Sign Up**.

תיאור כל מסכי הפרויקט

1. WelcomeActivity (מסך פתיחה)

• **תפקיד:** נקודת הכניסה לאפליקציה

• **תצוגה:** לוגו האפליקציה, שני כפתורים

• **כפתורים:**

◦ btnLogin — מעבר ל-LoginActivity

◦ btnRegister — מעבר ל-RoleSelectionActivity

2. RoleSelectionActivity (בחירת תפקיד בהרשמה)

• **תפקיד:** מציג שתי אפשרויות: Owner / Customer

• **כפתורים:**

◦ btnRoleOwner — פותח דיאלוג שמבקש קוד סודי (777)

◦ btnRoleCustomer — מעבר ישיר ל-RegisterActivity עם

תפקיד Customer

3. LoginActivity (מסך התחברות)

• **תפקיד:** קליטת שם משתמש/אימייל וסיסמה והתחברות

• **שדות:** etUsername, etPassword

• **כפתורים:** btnLogin, tvForgotPassword, tvRegisterLink

• **לוגיקה:** מאמת מול Room DB, אם הצליח מעדכן SessionManager

ומנתב לפי role

4. RegisterActivity (מסך הרשמה)

• **תפקיד:** יצירת חשבון חדש

• **שדות:** etFullName, etUsername, etEmail, etPhone

etPassword, etConfirmPassword, spinnerRole (מוסתר אם

תפקיד נשלח דרך Intent)

• **לוגיקה:** מצפין סיסמה ב-PBKDF2WithHmacSHA256, שומר

ב-Room, נרשם ל-Firebase Auth

5. SetupActivity (הקמת חשבון בעלים ראשון)

• **תפקיד:** מוצג רק כשאין משתמשים כלל בבסיס הנתונים

• **שדות:** etOwnerUsername, etOwnerPassword

etConfirmPassword

6. MainActivity (מסך ראשי לבעלים/מנהל/עובד)

- **תפקיד:** מסך הבית עם BottomNavigationView
- **טאבים:** Dashboard, Inventory, Sales, Tasks, More (תפריט נפתח)
- **תפריט More:** Purchases, Marketing, Reports, Seasons, Admin, Orders, Support, Logout
- **תוספת:** מפעיל AlarmManager לבדיקת מלאי

7. CustomerMainActivity (מסך ראשי ללקוח)

- **תפקיד:** מסך הבית של הלקוח
- **שורת עליונה:** לוגו + כפתור Sign Out (אייקון נעילה)
- **טאבים תחתונים:** Shop (Home), Cart, Orders, Favorites, Support

8. AddEditProductActivity (הוספת/עריכת מוצר)

- **תפקיד:** טופס ליצירה או עריכה של מוצר. **מוגבל ל-Owner בלבד**
- **שדות:** etProductName, etProductCategory, etProductSize, etProductColor, etProductQuantity, etProductPrice, etProductCostPrice
- **כפתור:** btnSaveProduct

9. AddSaleActivity (הוספת מכירה)

- **שדות:** מזהה מוצר, כמות, מחיר כולל, הערות

10. AddPurchaseActivity (הוספת רכישה מספק)

- **שדות:** מזהה מוצר, כמות, עלות כוללת, ספק, הערות

11. AddEditTaskActivity (משימה)

- **שדות:** כותרת, תיאור, מוקצה אל, סטטוס, עדיפות, תאריך יעד, פרטי

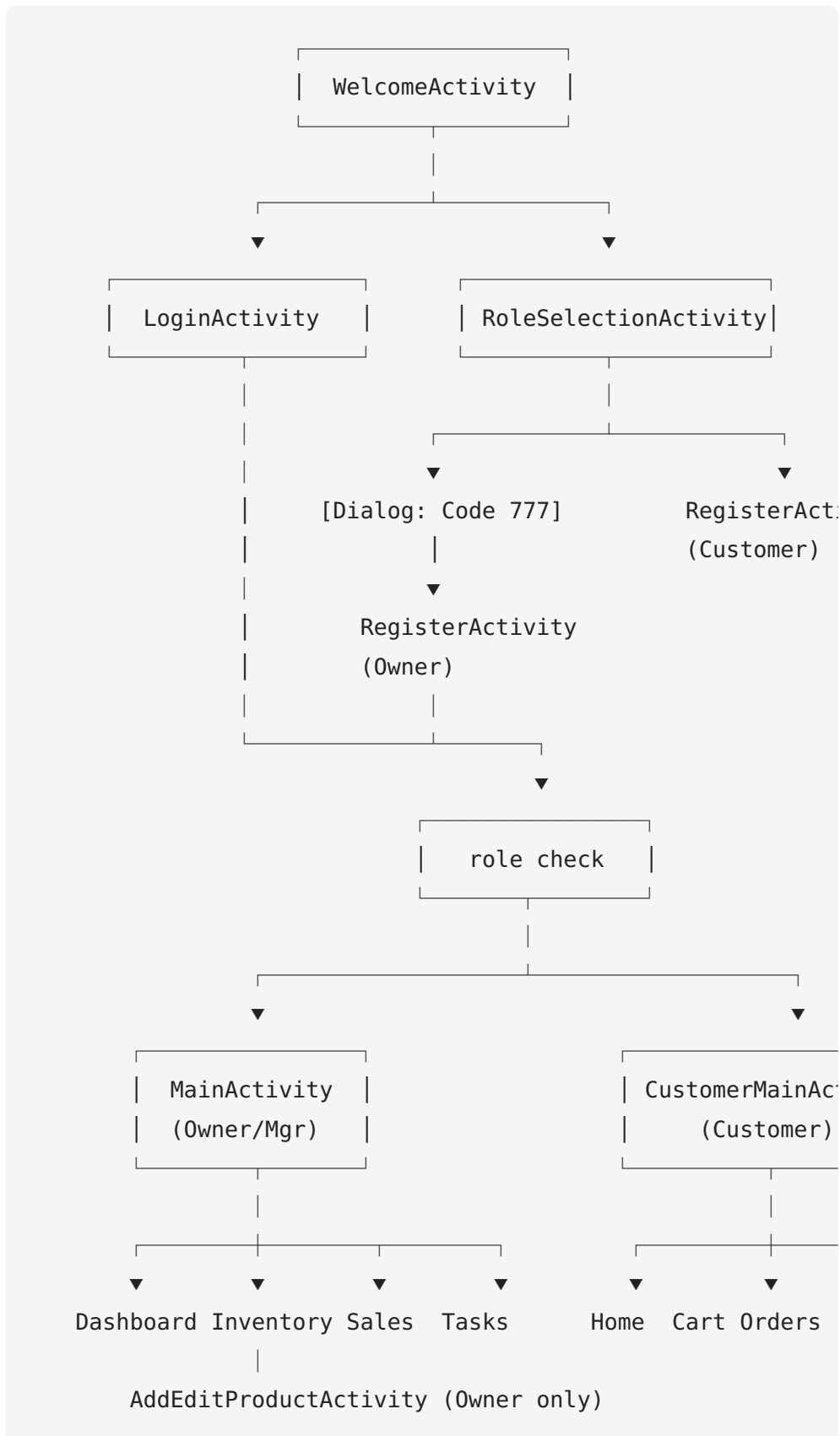
12. AddEditSeasonActivity (עונה)

• **שדות:** שם עונה, תאריך התחלה, סיום, התראה X ימים לפני

13. AddMetricActivity (מדד שיווקי)

• **שדות:** כותרת סרטון, פלטפורמה, צפיות, לייקים, שיתופים, תגובות

דיאגרמת זרימת מסכים (Screen Flow Diagram)



תיאור מחלקות הפרויקט (UML)

הארכיטקטורה בנויה על דפוס MVVM:



מחלקות תשתית עיקריות:

מחלקה	תפקיד
AppDatabase	Room Database singleton, מנהל את כל ה-DAOs
SessionManager	Singleton הזוכר את המשתמש המחובר ומבצע logout
PermissionManager	טבלת הרשאות סטטית לפי תפקיד
CryptoUtil	הצפנת סיסמאות עם PBKDF2WithHmacSHA256
FirestoreManager	מחלקה מרכזית לכל הפעולות מול Firestore
FirebaseAuthHelper	עטיפה ל-FirebaseAuth לפעולות /signIn/signOut
NotificationHelper	יצירת ערוץ התראות ושליחת התראת מלאי נמוך
LowStockReceiver	BroadcastReceiver שמופעל על ידי AlarmManager
StorePilotApp	Application class, מאתחל את Firebase ואת ערוץ ההתראות

מימוש הפרויקט

חלוקה בין לוגיקה ותצוגה

הפרויקט מקפיד על הפרדה מוחלטת בין השכבות: - **תצוגה (View)** — Activities ו-Fragments אחראיות רק על הצגת UI ועל קליטת אירועים. - **לוגיקה (ViewModel + Repository)** — כל העיבוד, האימותים, וחישובים מתבצעים בשכבת ה-ViewModel וה-Repository. - **גישת נתונים (DAO) +**

FirestoreManager — אך ורק בשכבה זו מתבצעות פעולות קריאה/כתיבה לבסיסי הנתונים.

תיאור המחלקות העיקריות

1. מחלקת **AppDatabase**

תפקיד: Singleton של בסיס הנתונים Room. מספקת גישה ל-12 DAOs שונים.

תכונות סטטיות: - `INSTANCE` — מופע יחיד של בסיס הנתונים - `dbExecutor` — `ExecutorService` עם 4 threads לביצוע פעולות DB ברקע

פעולות: - `getInstance(Context)` — מחזיר את ה-singleton ויוצר אותו במידת הצורך - DAOs: `userDao()`, `productDao()`, `taskDao()`, `saleDao()`, `purchaseDao()`, `orderDao()`, `orderItemDao()`, `cartDao()`, `favoriteDao()`, `supportMessageDao()`, `seasonDao()`, `videoMetricDao()`

```
@Database(entities = {User.class, Product.class, Sale.class,
    version = 1, exportSchema = true)
public abstract class AppDatabase extends RoomDatabase {
    public static final ExecutorService dbExecutor = Executors.newSingleThreadExecutor();
    private static AppDatabase INSTANCE;
    // ...
}
```

2. מחלקת **SessionManager**

תפקיד: ניהול המשתמש המחובר (Singleton, אחסון בזיכרון בלבד).

תכונות: - `instance` — סטטית, מופע יחיד - `loggedInUser` — מצביע למשתמש המחובר

פעולות: - `setLoggedInUser(User)` — מסמן משתמש כמחובר - `getLoggedInUser()` — מחזיר את המשתמש המחובר - `getUserRole()` — מחזיר את תפקיד המשתמש - `isLoggedIn()` — האם יש משתמש

מחובר - `logout()` — מאפס את המצב ומנתק מ-Firebase Authentication

3. מחלקת `PermissionManager`

תפקיד: טבלת הרשאות סטטית הקובעת איזה תפקיד יכול לבצע איזו פעולה.

תכונות סטטיות: - `permissionMap` — `Map<String, List<String>>`
המקשר בין הרשאה לרשימת תפקידים מורשים - קבועי הרשאות:
`VIEW_REPORTS`, `CREATE_SALE`, `MANAGE_PRODUCTS`, `MANAGE_USERS`
ועוד - קבועי תפקידים: `OWNER`, `STORE_MANAGER`, `SHIFT_MANAGER`, `EMPLOYEE`

פעולות: - `hasPermission(role, permission)` — בדיקה האם תפקיד מסוים מורשה - `currentUserHasPermission(permission)` — בדיקה עבור המשתמש המחובר

הגבלת הוספת מוצרים ל-Owner:

```
permissionMap.put(MANAGE_PRODUCTS, Arrays.asList(OWNER));
```

4. מחלקת `CryptoUtil`

תפקיד: הצפנת סיסמאות באמצעות אלגוריתם `.PBKDF2WithHmacSHA256`

פעולות: - `generateSalt()` — יוצר מלח אקראי באורך 16 בתים -
`hashPassword(password, salt)` — מצפין סיסמה עם המלח הנתון -
`verify(password, salt, hash)` — מאמת סיסמה מול hash שמור

5. מחלקת `FirestoreManager`

תפקיד: ניהול כל הפעולות מול `Firestore`. מפצלת את הנתונים ל-collections שונים.

פעולות עיקריות: - `saveUser(User)` — שומר/מעדכן משתמש ב- `users`
- `collection` - `saveProduct(Product)` — שומר/מעדכן מוצר -
- `deleteProduct(int)` — מוחק מוצר - `saveTask(Task)`
- `deleteTask(int)` - `saveSale(Sale)` , `saveOrder(Order)`
`savePurchase(Purchase)` , `updateOrderStatus(int, String)`

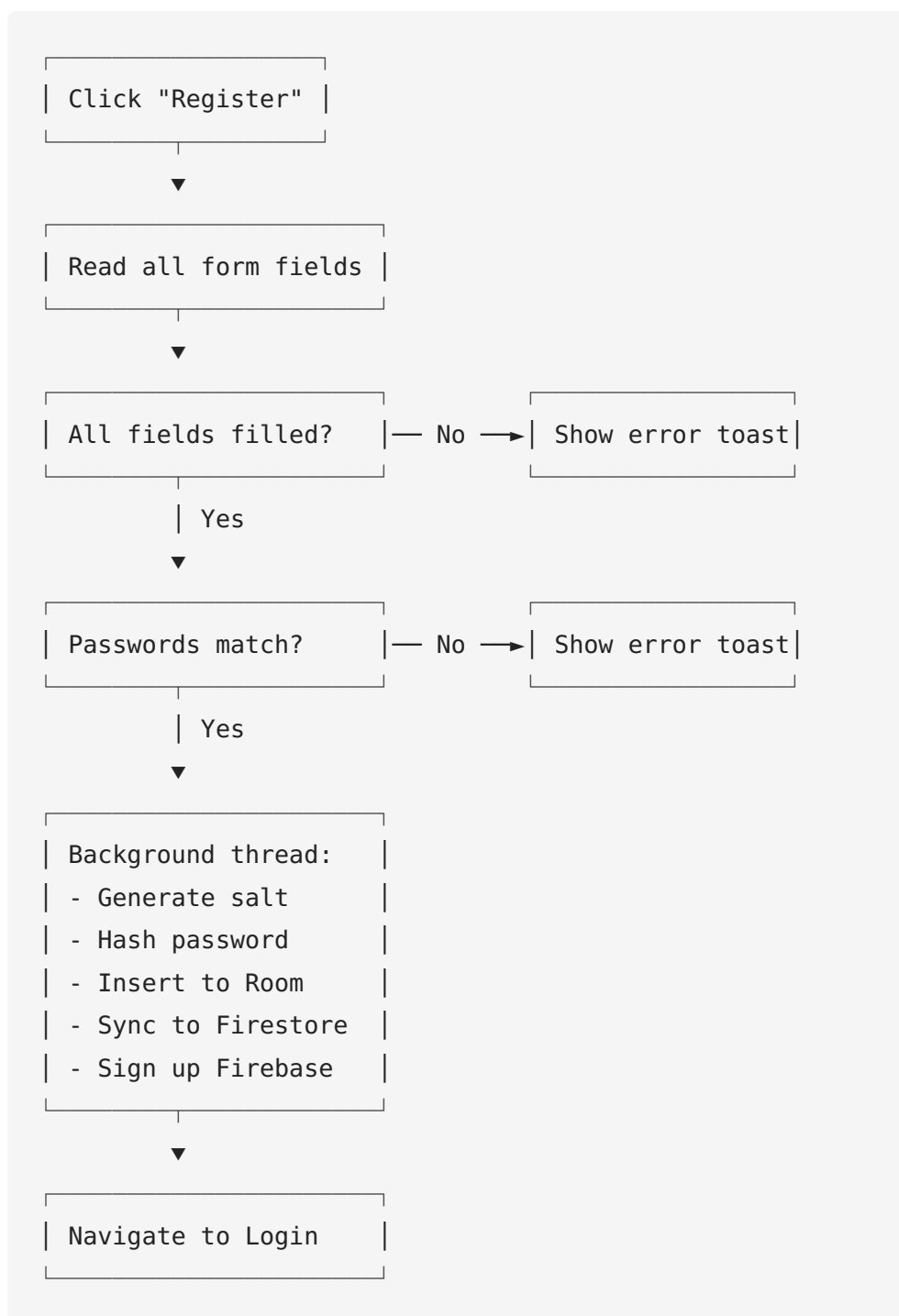
מבנה מסמך לדוגמה (`products/product_5`):

```
{
  "name": "T-Shirt",
  "category": "Clothing",
  "price": 49.90,
  "quantity": 25,
  "localId": 5,
  "createdAt": 1716000000000
}
```

6. מחלקת `RegisterActivity`

תפקיד: טופס יצירת חשבון חדש. כולל אימות שדות, הצפנת סיסמה, שמירה ל- Room ו- Firebase.

תרשים זרימה של פעולת `attemptRegister()`:



בסיס הנתונים

הפרויקט משתמש בשני בסיסי נתונים במקביל:

1. Room (SQLite) — מקומי:

טבלה	תיאור
users	משתמשי המערכת

טבלה	תיאור
products	מלאי
sales	מכירות
purchases	רכישות מספקים
orders	הזמנות לקוחות
order_items	פריטים בהזמנה
cart_items	פריטים בעגלה
favorites	מועדפים של לקוחות
tasks	משימות
seasons	עונות/קולקציות
video_metrics	מדדים שיווקיים
support_messages	הודעות צ'אט

2. Firestore (NoSQL) – ענן:

Collection	מסמכים
users	<username>
products	product_<id>
tasks	task_<id>
sales	sale_<date>_<productId>
orders	order_<id>
purchases	purchase_<date>_<productId>

3. Firebase Authentication: שומר רק email + password
 .displayName (= username)

פעולות על בסיסי הנתונים: - **קריאה:** דרך LiveData מ-Room (מהיר, מקומי, תומך offline) - **כתיבה:** Dual Write — קודם ל-Room ואז ל-Firebase (מתבצע ב-Background thread דרך FirebaseDatabase.dbExecutor) - **מחיקה:** באישור משתמש בלבד, נמחק משני המקורות

מדריך למשתמש

דרישות מערכת

- **גרסת אנדרואיד:** (Android 7.0 Nougat) API 24 ומעלה
- **גודל מסך:** מתאים לטלפונים וטאבלטים
- **חיבור אינטרנט:** נדרש לסנכרון Firebase (האפליקציה תעבוד גם offline על Room)
- **הרשאות נדרשות:**
 - INTERNET — לחיבור Firebase
 - POST_NOTIFICATIONS — להתראות מלאי (אנדרואיד +13)
 - ALARMEXACTCOMPLETED + SCHEDULEBOOTRECEIVE
- AlarmManager ל-7

גרסאות שנבדקו

- Android 13 (API 33) — Pixel 7 emulator
- Samsung — Android 11 (API 30) מכשיר פיזי
- emulator — Android 8.0 (API 26) לבדיקות תאימות

תהליך הפעלה ראשונה

1. **התקנה:** התקן את קובץ ה-APK או הרץ דרך Android Studio.
2. **הקמת בעלים ראשון:** אם זו ההפעלה הראשונה (אין משתמשים), המערכת תפנה ל-SetupActivity ליצירת חשבון הבעלים הראשי.
3. **כניסה רגילה:** מסך הפתיחה מציג Sign In / Sign Up.

הוראות שימוש

יצירת חשבון לקוח חדש:

1. ב-WelcomeActivity לחץ **Sign Up**
2. בחר **Customer**
3. מלא את הטופס ולחץ **Create Account**
4. תעבור למסך התחברות — הזן את הפרטים ולחץ **Sign In**

יצירת חשבון בעלים:

1. ב-WelcomeActivity לחץ **Sign Up**
2. בחר **Owner** — תפתח חלונית הקלדת קוד סודי
3. הזן את הקוד 777 ולחץ **Confirm**
4. מלא את הטופס בכלל שדותיו

הוספת מוצר חדש (Owner בלבד):

1. בכרטיסיית **Inventory** לחץ על הכפתור הצף +
2. מלא: שם, קטגוריה, גודל, צבע, כמות, מחיר, עלות
3. לחץ **Save**

ביצוע הזמנה (Customer):

1. בכרטיסיית **Shop** בחר מוצר → לחץ **Add to Cart**
2. עבור לכרטיסיית **Cart**, וודא את הפריטים
3. לחץ **Checkout** → בחר שיטת תשלום + כתובת
4. לחץ **Place Order** → תקבל מסך אישור

יציאה מהמערכת:

- **Customer**: בשורה העליונה של CustomerMainActivity לחץ על אייקון הנעילה → אשר ב-dialog
- **Owner/Manager**: מתפריט **More** בחר **Logout**

הודעות למשתמש (Alerts)

ההודעה	המקרה
"Please fill all required fields"	שדות חסרים בטופס
"Passwords do not match"	סיסמאות לא תואמות
"Password must be at least 6 characters"	סיסמה קצרה
".Incorrect code. Access denied"	קוד בעלים שגוי
X product(s) are running low. Tap to review." (Notification)	מלאי נמוך
".Only the Owner can add or edit products"	גישה למסך אסור
"!Order placed"	הזמנה בוצעה

מגבלות

- אורך סיסמה: לפחות 6 תווים
- שם משתמש: חייב להיות ייחודי במערכת
- אימייל: חייב להיות בפורמט תקני
- מחירים: חיוביים בלבד
- כמות במלאי: לא יכולה לרדת מתחת ל-0

סיכום אישי / רפלקציה

תיאור תהליך העבודה

עבדתי על פרויקט StorePilot במשך כשנה. התהליך התחיל מהרעיון הראשוני — שילוב של מערכת ניהול ואפליקציית קניות במקום אחד — והתפתח לפרויקט מורכב הכולל למעלה מ-90 קבצי Java ועשרות layouts.

הצלחות

- **ארכיטקטורת MVVM נקייה** — הצלחתי להפריד באופן מוחלט בין שכבות ה-View, ה-ViewModel וה-Repository, מה שאפשר תחזוקה קלה.
- **שילוב Firebase מוצלח** — דפוס ה-Dual Write (כתיבה ל-Room וגם ל-Firebase) פותר את בעיית ה-offline ובמקביל מסנכרן לענן.
- **מנגנון הרשאות** — טבלת PermissionManager גמישה ומאפשרת להוסיף בקלות הרשאות חדשות.
- **התראות אוטומטיות** — AlarmManager + BroadcastReceiver שעובדים בצורה אמינה.

אתגרים וקשיים

1. **נכבולת ב-LiveData observers** — בתחילה רשמתי observers בתוך observers אחרים, מה שיצר רישומים מרובים בכל עדכון. הפתרון: שני observers נפרדים שכל אחד שומר את הנתונים בשדה מקומי, ופונקציית מיזוג שמופעלת כשמתעדכן אחד מהם.
2. **שגיאת Material3 styles** — תמת האפליקציה היא MaterialComponents (M2) ולא Material3 (M3), והייתי צריך להעביר את כל ה-layouts ל-Widget.MaterialComponents.
3. **תקלות Gradle cache** — תיקיית transforms-3 השחיתה את עצמה מספר פעמים. למדתי את הפקודה `Remove-Item -Recurse -Force` לניקוי.

תהליך הלמידה

- למדתי לעומק על דפוסי עיצוב — Singleton, Observer, Repository, Adapter.
- הכרתי את Firebase מקצה לקצה — Authentication, Firestore, ההבדל בין NoSQL ל-SQL.
- שיפרתי את ההבנה שלי במנגנוני אבטחה — PBKDF2, salts, hashing.
- הבנתי את החיוניות של עבודה ב-background threads (ExecutorService) כדי לא לתקוע את ה-UI.

בראייה לאחור

אם הייתי מתחיל את הפרויקט מחדש: - הייתי משתמש ב-Hilt או Dagger ל-Dependency Injection מההתחלה במקום ליצור instances ידנית. - הייתי מטמיע Kotlin Coroutines במקום ExecutorService — תחביר נקי יותר. - הייתי מוסיף Unit Tests באופן שיטתי מהיום הראשון. - הייתי שוקל שימוש ב-Jetpack Compose במקום XML Views.

שיפורים עתידיים

- **Firebase Cloud Messaging** — התראות push אמיתיות מהשרת ולא רק מקומיות.
- **Image upload** — Firebase Storage לתמונות מוצרים.
- **Multi-store support** — תמיכה בכמה חנויות באותו account.
- **Analytics dashboard** — Firebase Analytics עם תרשימים.
- **תמיכה בשפות נוספות** — i18n מלא לעברית/אנגלית/ערבית.

תובנות אישיות

הפרויקט לימד אותי שיותר חשוב לתכנן היטב מאשר לקודד מהר. בכל פעם שדילגתי על שלב התכנון של המסכים והקלאסים, נאלצתי לחזור ולתקן בעלות גבוהה. למדתי גם את החשיבות של documentation — הערות Javadoc במקומות הנכונים שמרו אותי שפוי כשחזרתי לקוד אחרי כמה שבועות.

ביבליוגרפיה

1. Android Developers Documentation. Google Inc. (2024).
<https://developer.android.com/docs>
2. //:https. Firebase Documentation. Google Inc. (2024).
firebase.google.com/docs
3. //:https. Room Persistence Library Guide. Google Inc. (2024).
developer.android.com/jetpack/androidx/releases/room

4. Material Components for Android .Google Inc. (2024) .
-<https://github.com/material-components/material-components-android>
5. Java Cryptography Architecture .Oracle Corporation. (2024) .
[/https://docs.oracle.com/javase/8/docs/technotes/guides/security/crypto/CryptoSpec.html](https://docs.oracle.com/javase/8/docs/technotes/guides/security/crypto/CryptoSpec.html)
6. — NIST Special Publication 800-132 .NIST. (2017) .
.Recommendation for Password-Based Key Derivation
.National Institute of Standards and Technology
7. Patterns of Enterprise Application .Fowler, M. (2002) .
.Addison-Wesley Professional .Architecture
8. .Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994) .
Design Patterns: Elements of Reusable Object-Oriented
.Addison-Wesley .Software
9. <https://developer.android.com/studio/intro> .Android Studio User Guide .JetBrains s.r.o. (2024) .
10. Android development .Stack Overflow Community. (2024) .
<https://stackoverflow.com/questions/tagged/android> .Q&A
-

נספחים

נספח א' — קובץ `google-services.json`

ממוקם ב- `app/google-services.json` . מכיל את הגדרות החיבור
לפרויקט ID, PROJECT_IDKEY, APPFirebase: API

נספח ב' — מבנה תיקיות הפרויקט

```
StorePilot/
├── app/
│   ├── google-services.json
│   ├── build.gradle
│   └── src/main/
│       ├── AndroidManifest.xml
│       ├── java/com/storepilot/
│       │   ├── StorePilotApp.java
│       │   ├── MainActivity.java
│       │   ├── auth/ (קבצים 8)
│       │   ├── core/ (קבצים 8)
│       │   ├── customer/ (קבצים 15)
│       │   ├── manager/ (קבצים 5)
│       │   ├── admin/ (קבצים 2)
│       │   ├── dashboard/ (קובץ 1)
│       │   ├── inventory/ (קבצים 4)
│       │   ├── sales/ (קבצים 3)
│       │   ├── purchases/ (קבצים 3)
│       │   ├── tasks/ (קבצים 3)
│       │   ├── seasons/ (קבצים 3)
│       │   ├── marketing/ (קבצים 3)
│       │   ├── reports/ (קובץ 1)
│       │   ├── db/
│       │   │   ├── dao/ (קבצים 12)
│       │   │   └── entities/ (קבצים 12)
│       │   ├── repositories/ (קבצים 11)
│       │   └── viewmodels/ (קבצים 12)
│       └── res/
│           ├── layout/ (40+ XML files)
│           ├── drawable/ (אייקונים ועיצוב)
│           ├── values/ (strings, colors, themes)
│           └── menu/ (תפריטים)
├── build.gradle
└── settings.gradle
```

נספח ג' — סטטיסטיקות הפרויקט

• קבצי Java: 95+

- **Activities**: 12
- **Fragments**: 21
- **ViewModels**: 12
- **Repositories**: 11
- **Room Entities**: 12
- **Room DAOs**: 12
- **Firestore Collections**: 6
- **שורות קוד**: כ-7,000

נספח ד' — נושאים מתקדמים שיושמו

מיקום	הרחבה
21 fragments בכל האפליקציה	Fragment
— AppDatabase.dbExecutor New thread pool עם 4 threads	Threads (ExecutorService)
+ LowStockReceiver + NotificationHelper scheduleLowStockAlarm	+ AlarmManager Notification
+ FirestoreManager FirebaseAuthHelper	Firebase / Firestore (Remote DB)
,RoleSelectionActivity (owner code) CustomerMainActivity (logout)	AlertDialog
LowStockReceiver	BroadcastReceiver
כל פעולת DB ב-Repository רצה ב-dbExecutor	Async/Background work

נספח ה' — קוד מקור (Source Code)

קוד המקור המלא נשמר ב-GitHub: <https://github.com/ahmadhijazys2/StorePilot>

ההערות תועדו בסגנון Javadoc במקומות הקריטיים, ובהערות חד-שורתיות במקומות שבהם הצורך פחות מורכב.

דוגמה לתייעוד Javadoc במחלקת CryptoUtil:

```
/**
 * Hashes a plaintext password using PBKDF2WithHmacSHA256.
 *
 * @param password the plaintext password to hash
 * @param salt      the random salt to mix with the password
 * @return Base64-encoded hash string
 */
public static String hashPassword(String password, String salt)
```

סוף ספר הפרויקט